03. Reth 스토리지 시스템 분석

개요

Reth의 스토리지 시스템은 Ethereum 블록체인 데이터를 효율적으로 저장하고 관리하기 위한 계층화된 아키텍처를 제공합니다. 이 시스템은 MDBX 데이터베이스를 기반으로 하며, 압축된 데이터 형식과 Merkle Patricia Trie를 통해 최적화된 성능을 제공합니다.

전체 아키텍처

```
┌─────────────────────────────────────────────────────────────┐
│ Provider Layer │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────┐ │
│ │ ProviderFactory │ │ DatabaseProvider│ │ StateProvider│ │
│ └──────────────┘ └──────────────┘ └──────────┘ │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│ Storage API Layer │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────┐ │
│ │   BlockReader   │ │  AccountReader  │ │ ReceiptProvider│ │
│ └──────────────┘ └──────────────┘ └──────────┘ │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
```

```
| Trie System |
| ┌──────────────────┐ ┌──────────────────┐ ┌──────────────┐ |
| |    StateRoot    | |    StorageRoot  | |  SparseTrie | |
| └──────────────────┘ └──────────────────┘ └──────────────┘ |
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
| Database Layer |
| ┌──────────────────┐ ┌──────────────────┐ ┌──────────────┐ |
| |    MDBX Core    | |      Codecs     | | Static Files| |
| └──────────────────┘ └──────────────────┘ └──────────────┘ |
└──────────────────────────────────────────────────────────────┘
```

핵심 컴포넌트 분석

1. Storage 디렉터리 구조 분석

1.1 `crates/storage/` 하위 디렉터리 역할

`db/` - 데이터베이스 추상화 계층

- 역할: 데이터베이스 트랜잭션 및 커서 관리
- 핵심 구조체: `Database`, `DbTx`, `DbTxMut`
- 주요 기능: 트랜잭션 생성, 커밋, 롤백

`db-api/` - 데이터베이스 API 정의

- 역할: 데이터베이스 인터페이스 및 테이블 정의

- 핵심 구조체: `Table`, `DbCursorRO`, `DbCursorRW`
- 주요 기능: 테이블 스키마 정의, 커서 인터페이스

`db-common/` - 공통 데이터베이스 유틸리티

- 역할: 데이터베이스 간 공통 기능 제공
- 핵심 기능: 공통 타입 정의, 유틸리티 함수

`db-models/` - 데이터 모델 정의

- 역할: 데이터베이스에 저장되는 데이터 구조 정의
- 핵심 구조체: `AccountBeforeTx`, `StoredBlockBodyIndices`
- 주요 기능: 데이터 직렬화/역직렬화

`provider/` - 데이터 접근 계층

- 역할: 상위 레벨 데이터 접근 인터페이스 제공
- 핵심 구조체: `ProviderFactory`, `DatabaseProvider`
- 주요 기능: 데이터 읽기/쓰기 추상화

`libmdbx-rs/` - MDBX 데이터베이스 바인딩

- 역할: MDBX 데이터베이스와의 저수준 인터페이스
- 핵심 구조체: `Environment`, `Transaction`, `Cursor`
- 주요 기능: MDBX C 라이브러리 래핑

`codecs/` - 데이터 압축 및 인코딩

- 역할: 효율적인 데이터 저장을 위한 압축 알고리즘
- 핵심 트레이트: `Compact`, `Encode`, `Decode`
- 주요 기능: 데이터 압축, 가변 길이 인코딩

`nippy-jar/` - 압축된 정적 파일 관리

- 역할: 블록 데이터의 압축 저장 및 관리

- 주요 기능: 정적 파일 압축, 빠른 읽기 접근

`static-file/` - 정적 파일 시스템

- 역할: 불변 블록 데이터의 효율적 저장
- 핵심 구조체: `StaticFileProvider`, `StaticFileWriter`
- 주요 기능: 헤더, 트랜잭션, 영수증 정적 저장

`zstd-compressors/` - Zstandard 압축

- 역할: 트랜잭션 및 영수증 데이터 압축
- 주요 기능: 사전 기반 압축, 압축률 최적화

2. Trie 시스템 구조 분석

2.1 `crates/trie/` 디렉터리 구조

`trie/` - 핵심 Merkle Patricia Trie 구현

- 역할: 상태 루트 계산 및 증명 생성
- 핵심 구조체: `StateRoot`, `StorageRoot`
- 주요 기능: 트라이 구성, 해시 계산, 증명 생성

`db/` - 데이터베이스 통합 트라이

- 역할: 데이터베이스와 트라이 시스템 연결
- 핵심 구조체: `DatabaseStateRoot`, `DatabaseTrieCursor`
- 주요 기능: 데이터베이스 기반 트라이 탐색

`sparse/` - 희소 트라이 구현

- 역할: 메모리 효율적인 트라이 구조
- 핵심 구조체: `SparseTrie`, `SparseTrieState`
- 주요 기능: 부분 트라이 구성, 증분 업데이트

`common/` - 트라이 공통 유틸리티

- 역할: 트라이 시스템 간 공통 기능
- 주요 기능: 니블 처리, 해시 유틸리티

`parallel/` - 병렬 트라이 처리

- 역할: 멀티스레드 트라이 계산
- 주요 기능: 병렬 상태 루트 계산

데이터 흐름 분석

3. ProviderFactory → DatabaseProvider → MDBX 호출 체인

3.1 ProviderFactory 생성 및 설정

```
// ProviderFactory 생성
let factory = ProviderFactory::new(
db, // MDBX 데이터베이스 인스턴스
chain_spec, // 체인 스펙
static_file_provider, // 정적 파일 프로바이더
);

// 읽기 전용 프로바이더 생성
let provider = factory.provider()?;
```

3.2 DatabaseProvider 트랜잭션 관리

```
impl<N: ProviderNodeTypes> ProviderFactory<N> {
pub fn provider(&self) -> ProviderResult<DatabaseProviderRO<N::DB, N>> {
Ok(DatabaseProvider::new(
self.db.tx()?, // MDBX 읽기 트랜잭션 생성
self.chain_spec.clone(),
self.static_file_provider.clone(),
self.prune_modes.clone(),
self.storage.clone(),
))
}
}
```

3.3 실제 MDBX 호출 과정

```
// 1. 트랜잭션 생성 (libmdbx-rs)
let tx = env.begin_ro_txn()?;

// 2. 커서 생성
let mut cursor = tx.cursor_read::<tables::Headers>()?;

// 3. 데이터 조회
let header = cursor.seek_exact(block_number)?;
```

```
// 4. 데이터 디코딩
let decoded_header = Header::decode(&header_bytes)?;
```

데이터 저장 형식 분석

4. 인코딩 및 압축 시스템

4.1 Compact 인코딩

Reth는 `Compact` 트레이트를 통해 효율적인 데이터 압축을 구현합니다:

```
pub trait Compact: Sized {
fn to_compact<B>(&self, buf: &mut B) -> usize
where
B: bytes::BufMut + AsMut<[u8]>;
fn from_compact(buf: &[u8], len: usize) -> (Self, &[u8]);
}
```

압축 전략:

- 정수형: 선행 0 바이트 제거
- 옵션형: 존재 여부를 비트플래그로 표현
- 가변 길이 정수: varint 인코딩 사용
- 고정 크기 배열: 압축 없이 직접 저장

4.2 RLP (Recursive Length Prefix) 인코딩

Ethereum 표준 RLP 인코딩을 사용하여 블록 헤더 및 트랜잭션 데이터를 저장합니다:

```
// 블록 헤더 RLP 인코딩
let encoded_header = header.encode();


// 트랜잭션 RLP 인코딩
let encoded_tx = transaction.encode_2718();
```

5. 데이터베이스 테이블 구조

5.1 주요 테이블 분석

Headers 테이블

- 키: `BlockNumber` (u64)
- 값: `Header` (RLP 인코딩된 블록 헤더)
- 용도: 블록 헤더 저장

Transactions 테이블

- 키: `TxNumber` (u64)
- 값: `TransactionSigned` (서명된 트랜잭션)
- 용도: 정규 트랜잭션 저장

PlainAccountState 테이블

- 키: `Address` (20바이트 주소)
- 값: `Account` (계정 상태)
- 용도: 현재 계정 상태 저장

PlainStorageState 테이블 (DupSort)

- 키: `Address` (계정 주소)

- 서브키: `B256` (스토리지 키)
- 값: `StorageEntry` (스토리지 값)
- 용도: 계정별 스토리지 상태

HashedAccounts 테이블

- 키: `B256` (keccak256(address))
- 값: `Account` (계정 상태)
- 용도: 머클 트리 계산용 해시된 계정

5.2 히스토리 테이블

AccountsHistory 테이블

- 키: `ShardedKey<Address>` (샤드된 주소 키)
- 값: `BlockNumberList` (블록 번호 목록)
- 용도: 계정 변경 이력 추적

StoragesHistory 테이블

- 키: `StorageShardedKey` (샤드된 스토리지 키)
- 값: `BlockNumberList` (블록 번호 목록)
- 용도: 스토리지 변경 이력 추적

5.3 트라이 테이블

AccountsTrie 테이블

- 키: `StoredNibbles` (저장된 니블)
- 값: `BranchNodeCompact` (압축된 브랜치 노드)
- 용도: 계정 머클 트리 노드 저장

StoragesTrie 테이블 (DupSort)

- 키: `B256` (해시된 주소)
- 서브키: `StoredNibblesSubKey` (니블 서브키)
- 값: `StorageTrieEntry` (스토리지 트라이 엔트리)
- 용도: 스토리지 머클 트리 노드 저장

성능 최적화 기법

6. 샤딩 (Sharding) 시스템

Reth는 히스토리 데이터를 효율적으로 관리하기 위해 샤딩 시스템을 사용합니다:

```
pub struct ShardedKey<T> {
pub key: T,
pub highest_block_number: BlockNumber,
}
```

샤딩 전략:

- 각 샤드는 특정 블록 범위의 변경사항을 포함
- 마지막 샤드는 `u64::MAX` 를 사용하여 현재 상태 표시
- 이진 검색을 통한 빠른 히스토리 조회

7. 정적 파일 시스템

불변 데이터(헤더, 트랜잭션, 영수증)는 정적 파일로 별도 저장:

```
pub struct StaticFileProvider<P: NodePrimitives> {
// 정적 파일 관리
```

```
}
```

장점:

- 압축률 향상 (Zstandard 사전 압축)
- 빠른 순차 읽기 성능
- 데이터베이스 크기 감소

8. 메모리 최적화

8.1 Compact 인코딩 최적화

```
impl Compact for U256 {
fn to_compact<B>(&self, buf: &mut B) -> usize {
let inner = self.to_be_bytes::<32>();
let size = 32 - (self.leading_zeros() / 8);
buf.put_slice(&inner[32 - size..]);
size
}
}
```

8.2 희소 트라이 최적화

```
pub struct SparseTrie {
// 변경된 노드만 메모리에 유지
// 미변경 노드는 데이터베이스에서 지연 로딩
```

```
}
```

트랜잭션 관리

9. ACID 속성 보장

원자성 (Atomicity)

- MDBX 트랜잭션을 통한 모든 변경사항 원자적 커밋
- 실패 시 자동 롤백

일관성 (Consistency)

- 체인 스펙 검증을 통한 데이터 무결성
- 머클 루트 검증을 통한 상태 일관성

격리성 (Isolation)

- MVCC (Multi-Version Concurrency Control) 지원
- 읽기 트랜잭션 간 격리 보장

지속성 (Durability)

- MDBX의 WAL (Write-Ahead Logging)
- 시스템 크래시 시 데이터 복구 보장

10. 동시성 제어

```
impl<N: ProviderNodeTypes> ProviderFactory<N> {
// 읽기 전용 프로바이더 (동시 접근 가능)
pub fn provider(&self) -> ProviderResult<DatabaseProviderRO<N::DB, N>> {
// 여러 읽기 트랜잭션 동시 실행 가능
```

```
}

// 읽기-쓰기 프로바이더 (배타적 접근)

pub fn provider_rw(&self) -> ProviderResult<DatabaseProviderRW<N::DB, N>> {

// 단일 쓰기 트랜잭션만 허용

}

}
```

오류 처리 및 복구

11. 오류 타입 계층

```
pub enum ProviderError {

Database(DatabaseError),

StaticFile(StaticFileError),

Trie(TrieError),

// ... 기타 오류 타입

}
```

12. 데이터 무결성 검증

```
// 상태 루트 검증

if computed_state_root != expected_state_root {

return Err(ProviderError::StateRootMismatch);
```

```
}

// 머클 증명 검증
if !verify_merkle_proof(&proof, &key, &value, &root) {
return Err(ProviderError::InvalidMerkleProof);
}
```

결론

Reth의 스토리지 시스템은 다음과 같은 특징을 가집니다:

1. **계층화된 아키텍처**: Provider → API → Trie → Database 계층 분리
2. **효율적인 압축**: Compact 인코딩과 Zstandard 압축을 통한 저장 공간 최적화
3. **고성능 데이터베이스**: MDBX를 통한 빠른 읽기/쓰기 성능
4. **샤딩 시스템**: 히스토리 데이터의 효율적 관리
5. **정적 파일 시스템**: 불변 데이터의 별도 저장을 통한 성능 향상
6. **강력한 일관성**: 머클 트리와 ACID 속성을 통한 데이터 무결성 보장

이러한 설계를 통해 Reth는 Ethereum 메인넷의 대용량 데이터를 효율적으로 처리하면서도 높은 성능과 안정성을 제공합니다.